INTERNSPARK CYBERSECURITY INTERNSHIP

Vulnerability Assessment Report

Web Application Security Basics (DVWA)

Task 2: Web Application Security Basics

Internship provider	InternSpark
Task	Task 2 - Web Application Security Basics
Application assessed	DVWA (Damn Vulnerable Web Application), security level Low
Assessment environment	Local lab, http://127.0.0.1:4280/ (author's own machine)
Prepared by	Raghav Marda
Assessment date	23 October 2026 (from screenshot timestamps)

Table of Contents

1. Executive Summary 3

2. Introduction 3

3. Scope and Authorization 4

4. Tools and Technologies 4

5. Methodology 4

6. Detailed Vulnerability Findings 5

7. Burp Suite Request Analysis 9

8. Remediation Recommendations 10

9. Limitations 10

10. Conclusion 11

11. References 11

Appendix A: Evidence Register 11

1. Executive Summary

This report documents a web application security assessment carried out for InternSpark Cybersecurity Internship Task 2 (Web Application Security Basics). The target was DVWA, an intentionally vulnerable training application, running in a local lab at `http://127.0.0.1:4280/` with its security level set to Low. The assessment was authorized, since DVWA was running on the author's own machine, and did not involve any public or production system.

Three activities were performed: a SQL Injection test on the DVWA SQL Injection module, a Reflected Cross-Site Scripting (XSS) test on the DVWA XSS (Reflected) module, and manual HTTP request interception and parameter inspection of the SQL Injection page using Burp Suite Community Edition. The tools used were DVWA, Google Chrome and Burp Suite Community Edition.

ID	Finding	Endpoint (local)	Indicative severity	Status
F-01	SQL Injection	/vulnerabilities/sqli/	High	Confirmed (Fig. 1)
F-02	Reflected Cross-Site Scripting	/vulnerabilities/xss_r/	Medium	Confirmed (Fig. 2, 3)

Overall conclusion. Both vulnerabilities were confirmed in the DVWA Low-security lab. This is expected behaviour for a training application and is not evidence about any other application. Severity ratings are indicative qualitative ratings, not formal CVSS scores. Burp Suite was used only for manual request inspection; no automated scanning was performed. The report also gives remediation guidance and a retest procedure for each finding.

2. Introduction

2.1 Background

Web applications accept input from users and use it to build database queries and web pages. When that input is not handled safely, attackers can change the behaviour of the application. SQL Injection and Cross-Site Scripting are two of the most widely taught examples of this class of weakness and are standard starting points for learning web application security.

2.2 Purpose of testing

The purpose of Task 2 was to gain hands-on experience identifying and demonstrating basic web application vulnerabilities in a safe environment, to use an intercepting proxy to examine client-server traffic, and to document the results in a structured report with remediation advice.

2.3 About DVWA

DVWA (Damn Vulnerable Web Application) is a deliberately insecure web application built for security training. It provides modules for common vulnerability classes (the modules visible in the screenshots include Brute Force, Command Injection, CSRF, File Inclusion, File Upload, SQL Injection, XSS and others) and a selectable security level. At the Low level its protections are minimal by design. It must only be run in an isolated, trusted environment, as it was here.

3. Scope and Authorization

Environment	Local lab on the author's own machine. Not a public or production website.
Base URL	`http://127.0.0.1:4280/`
Security level	Low (as set by the author; the level is not visible in the supplied screenshots).
Authorization	DVWA was installed and run by the author, who was authorized to test it.
Modules assessed	SQL Injection: `/vulnerabilities/sqli/` XSS (Reflected): `/vulnerabilities/xss_r/`
Tools used	DVWA, Google Chrome, Burp Suite Community Edition (v2026.9.2 shown in screenshots)

3.1 Exclusions

- Any public, production or third-party system.
- Other DVWA modules (for example Stored XSS, DOM XSS, Blind SQL Injection, Command Injection, CSRF, File Upload) and security levels other than Low.
- Burp Scanner or any automated or active scanning, Burp Intruder attacks, and Burp Repeater testing.
- Database enumeration, data modification, or exploitation beyond the inputs listed in Section 5.
- Source-code review of DVWA, and any claim of a comprehensive penetration test.

4. Tools and Technologies

Tool	Purpose in this assessment
DVWA	Intentionally vulnerable PHP-based web application used as the safe, local target.
Google Chrome	Web browser used to interact with DVWA, submit test inputs and observe results, including the JavaScript alert.
Burp Suite Community Edition	Intercepting proxy used to capture and inspect the HTTP request sent when interacting with the SQL Injection page. Used manually through Proxy > Intercept only.

5. Methodology

Testing followed this sequence. Each step was performed manually in the local lab.

Step	Activity	Detail
1	SQL Injection baseline	Submitted the normal user ID `1` in the SQL Injection module and observed the corresponding single database record. The request for this submission was captured in Burp (Figure 5).
2	SQL Injection test	Submitted `1' OR '1'='1' #` as the User ID and observed that multiple records were returned (Figure 1).
3	Reflected XSS test	Submitted `<script>alert('XSS')</script>` in the name field of the XSS (Reflected) module and observed the browser alert (Figures 2 and 3).
4	Burp request interception	With Burp Proxy interception on, intercepted the HTTP requests made while using the SQL Injection page and inspected the method, path and query parameters (Figures 4 and 5). Interception was then turned off and the page was allowed to load normally.

Step	Activity	Detail
5	Evidence capture	Screenshots were taken of each result. Their timestamps are listed in Appendix A.

6. Detailed Vulnerability Findings

Severity ratings below are **indicative** qualitative ratings for an intentionally vulnerable DVWA Low-security lab. They are not CVSS scores and are not a statement about any production system. The observed result in each finding is taken from the screenshots; impact, root cause and remediation are the assessor's technical analysis.

6.1 Finding F-01: SQL Injection

Finding ID	F-01
Vulnerability	SQL Injection (input alters SQL query logic)
Affected endpoint	`http://127.0.0.1:4280/vulnerabilities/sqli/`, parameter `id` (GET)
Severity (indicative)	**High.** A single crafted input made the application return records beyond the one intended, showing that user input controls the database query. In a real application, SQL Injection of this kind commonly allows broad data exposure. Further exploitation was not tested, so no higher rating is claimed.
Status	Confirmed in the local DVWA Low-security lab.

Description

The User ID field is used by the application to look up one user record. Instead of treating the input as a plain value, the application allowed the input to change the logic of the underlying SQL query. A quote character ended the intended value, `OR '1'='1'` added a condition that is always true, and `#` commented out the rest of the original statement (the # comment syntax is used by MySQL and MariaDB; the database engine was not confirmed in the screenshots). With the condition always true, the query matches every row in the table rather than one.

Illustrative query structure, shown only to explain the behaviour (the DVWA source was not reviewed in this assessment):

```
Intended:   SELECT first_name, last_name FROM users WHERE user_id = '1';
Injected:   SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1' #';
```

Technical evidence and observed result

- Baseline input `1` was submitted first and returned its corresponding record (the Burp capture of this request is Figure 5; a screenshot of the baseline result was not supplied).
- Test input `1' OR '1'='1' #` returned five records, shown as: admin admin; Gordon Brown; Hack Me; Pablo Picasso; Bob Smith (Figure 1).
- The URL in Figure 1 shows the input URL-encoded in the `id` parameter of a GET request: `/vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271%27+%23&Submit=Submit`

Reproduction steps performed

- 1. Opened the DVWA SQL Injection page in Chrome (Low security level).
- 2. Entered `1` in User ID and clicked Submit; observed one record.
- 3. Entered `1' OR '1'='1' #` in User ID and clicked Submit.
- 4. Observed multiple user records displayed (Figure 1).

Potential security impact

In a real application with the same weakness, an attacker could read data the application never intended to expose, and depending on the database permissions and query context could potentially read other tables or alter data. Only the retrieval of multiple name records was demonstrated here; nothing further was attempted.

Root cause

Based on the observed behaviour, the application most likely builds the SQL statement by inserting user input directly into the query string without parameterization or safe handling. This is inferred from behaviour, not from a source-code review.

Recommended remediation

- Use parameterized queries (prepared statements) so input is always treated as data, never as SQL. In PHP this means PDO or mysqli prepared statements.
- Do not build SQL by string concatenation with user input.
- Validate input as an additional control; for example, accept only integers for a numeric ID.
- Run the application's database account with least privilege (only the tables and operations it needs).
- Avoid showing detailed database errors to users.

Suggested validation / retest

After the fix, repeat the baseline and test inputs on the same endpoint. Expected result: 1 returns exactly one record, and the injection string returns no extra records (either no result or a handled validation error). Add this case to automated regression tests, and optionally compare against DVWA's higher security levels, which demonstrate stronger implementations.

Screenshot evidence

Figure 1 (01_sql_injection_proof.png), Figure 5 (05_burp_sqli_parameters.png).

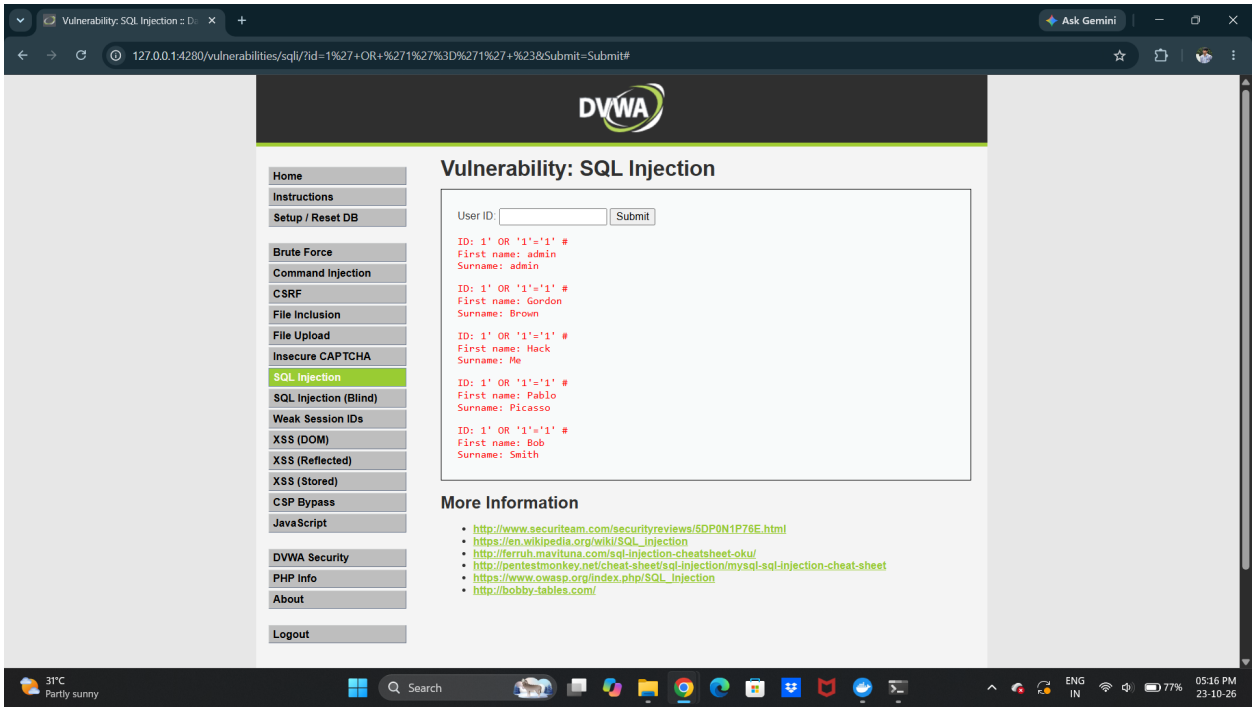


Figure 1: DVWA SQL Injection result showing multiple records (01_sql_injection_proof.png)

6.2 Finding F-02: Reflected Cross-Site Scripting (XSS)

Finding ID	F-02
Vulnerability	Reflected Cross-Site Scripting (XSS)
Affected endpoint	`http://127.0.0.1:4280/vulnerabilities/xss_r/`, parameter `name` (GET)
Severity (indicative)	**Medium.** JavaScript supplied in a request was executed in the browser, which is a confirmed XSS. Reflected XSS normally requires a victim to open a crafted link, which limits exposure compared with stored XSS. Real-world impact depends on the application's session handling and data, which were not assessed.
Status	Confirmed in the local DVWA Low-security lab.

Description

The name entered in the form is returned in the server's response. The application did not encode the value for the HTML output context, so the browser treated the `<script>` element as code rather than as text and executed it. Because the value travels in the URL query string, the same request can be delivered as a link.

Technical evidence and observed result

- Payload entered in the name field: `<script>alert('XSS')</script>`
- Figure 2 shows the browser dialog "127.0.0.1:4280 says XSS", confirming that the injected JavaScript executed. The tab shows the page still loading, consistent with the dialog pausing the page.
- Figure 3 shows the resulting page, with the payload visible in the URL (`?name=<script>alert%28%27XSS%27%29<%2Fscript>#`). The response area shows only "Hello", with no script text displayed, which is consistent with the payload being interpreted as markup rather than shown as text. Figure 3 by itself does not prove execution; Figure 2 does.

Reproduction steps performed

- 1. Opened the DVWA XSS (Reflected) page in Chrome (Low security level).
- 2. Entered the payload above in "What's your name?" and clicked Submit.
- 3. Observed the JavaScript alert containing the text XSS (Figure 2).

Potential security impact

In an application with the same flaw, an attacker could run script in a victim's browser within the application's origin, for example to alter page content, present fake forms to steal credentials, or act on the victim's behalf. Which of these is possible depends on the application and was not tested. Only a harmless alert was used here.

Root cause

User-supplied input is reflected into the HTML response without context-appropriate output encoding. This is inferred from the observed behaviour, not from a source-code review.

Recommended remediation

- Apply context-appropriate output encoding to all untrusted data (for HTML body content, HTML-encode special characters; in PHP, `htmlspecialchars` with a suitable flag and UTF-8 encoding).
- Use safe templating practices or frameworks that auto-escape output by default.
- Validate input where appropriate (for example, limit allowed characters for a name field), as an additional control only.
- Add a suitable Content Security Policy as defense in depth, and set the HttpOnly flag on session cookies.

Suggested validation / retest

After the fix, submit the same payload. Expected result: no alert appears and the characters are shown as literal text (or rejected). Check the page source to confirm the characters are encoded, and add the case to regression tests. Test other output contexts and parameters as well, not only this field.

Screenshot evidence

Figure 2 (02_reflected_xss_alert.png), Figure 3 (03_reflected_xss_response.png).

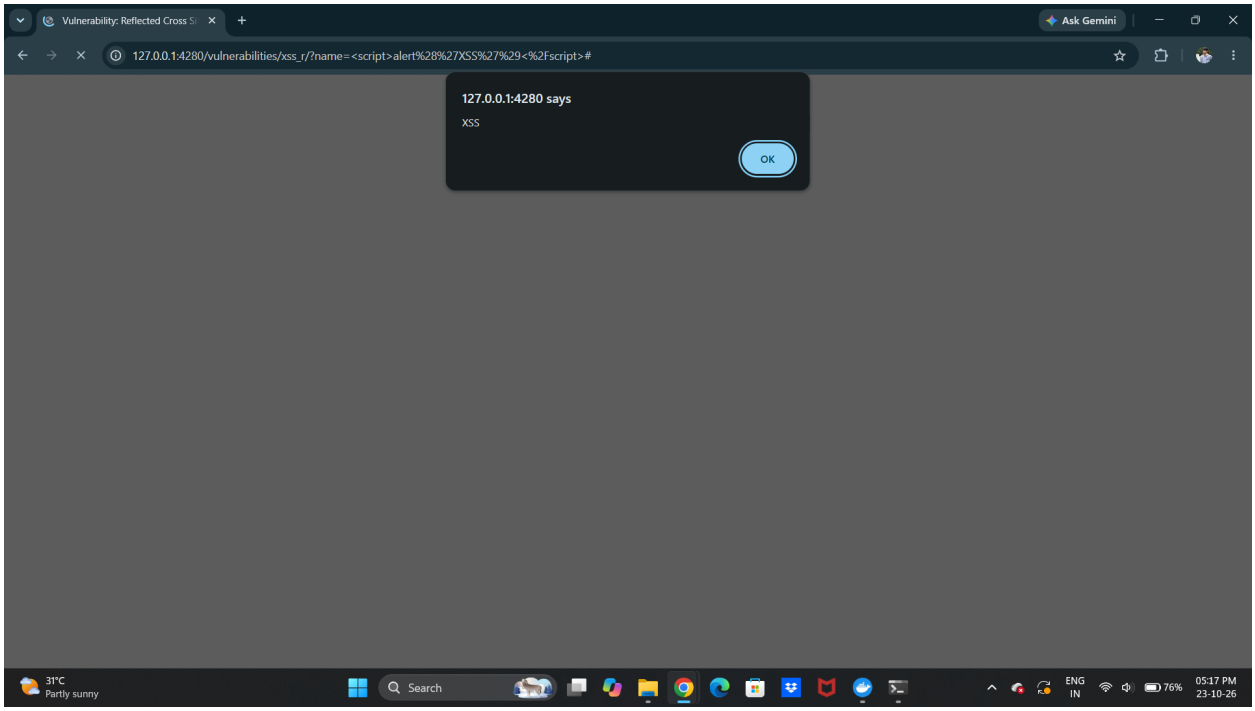


Figure 2: Browser alert showing XSS after submitting the payload (02_reflected_xss_alert.png)

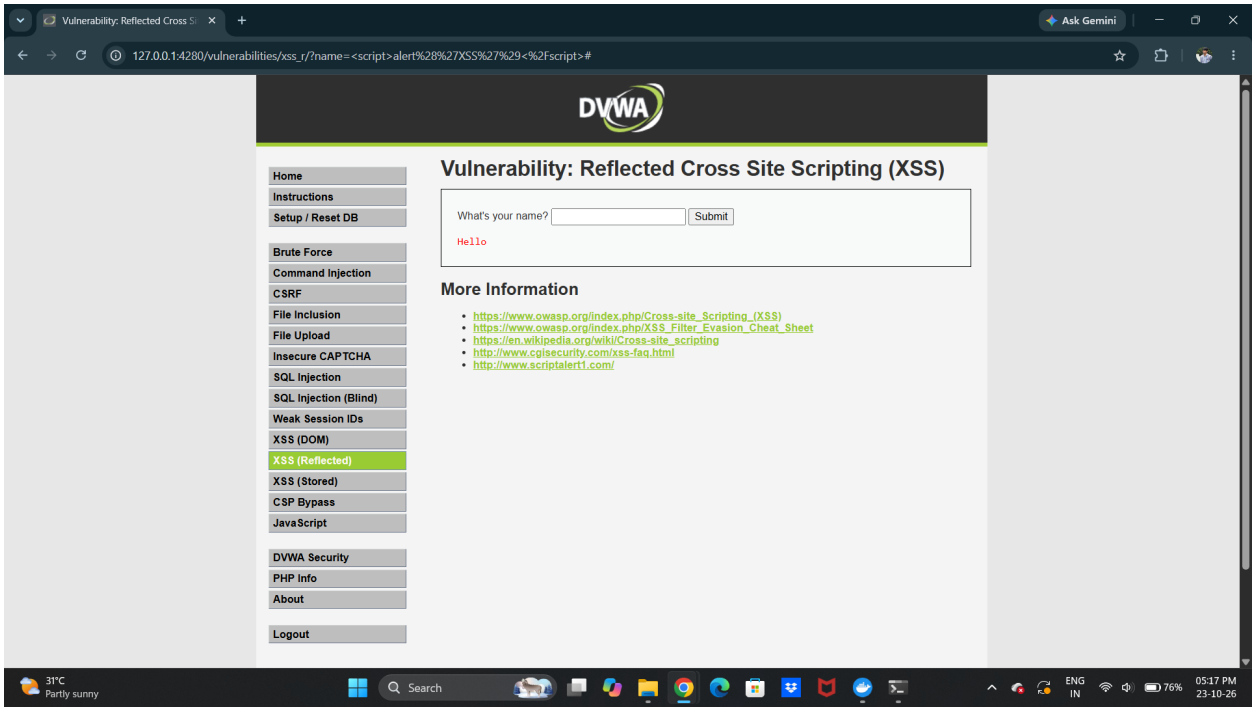


Figure 3: DVWA Reflected XSS page with payload in the URL (03_reflected_xss_response.png)

7. Burp Suite Request Analysis

7.1 The intercepted requests

Burp Suite Community Edition (Temporary Project) was used with Proxy interception on. Two requests to `http://127.0.0.1:4280` were captured, both HTTP GET:

Capture	Time (Burp)	Request line	Query parameters (Inspector)
Figure 4 (04_burp_request_interception.png)	17:33:01, 23 Oct	`GET /vulnerabilities/sqli/ HTTP/1.1`	0 (initial page load; no parameters)
Figure 5 (05_burp_sqli_parameters.png)	17:35:42, 23 Oct	`GET /vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1`	2: id = 1, Submit = Submit

The Raw/Pretty view in both captures also shows standard browser headers, including `Host: 127.0.0.1:4280`, `Accept-Language: en-US,en;q=0.9`, `Upgrade-Insecure-Requests: 1`, a Chrome-based `User-Agent` on Windows, and `Sec-Fetch-Site: same-origin`, `Sec-Fetch-Mode: navigate` and `Sec-Fetch-Dest: document`. The Inspector reports 2 request cookies in Figure 5 (their names and values are not visible in the screenshot) and 16 request headers in Figure 4.

7.2 The id and Submit parameters

Parameter	Observed value	Role
`id`	`1`	The User ID entered in the form. It is the value the application uses to look up a record, and the value manipulated in F-01.
`Submit`	`Submit`	Sent by the form's Submit button to indicate the form was submitted. It does not carry user-entered data.

Because the form uses GET, both values appear in the URL and are visible, editable and loggable. This is also why the SQL injection input in Figure 1 and the XSS payload in Figures 2 and 3 appear in the browser address bar.

7.3 Value of an intercepting proxy

An intercepting proxy sits between the browser and the server. Burp Suite's Proxy can hold a request before it reaches the server so the tester can see exactly what the browser sends (method, path, parameters, headers, cookies), and can then forward or drop it. This shows what the browser interface hides, helps identify which inputs reach the server, and is the basis for later manual testing. Interception was turned off afterwards, and the page loaded normally, as is standard Burp practice.

7.4 Request inspection versus an automated scan

What was done here is manual request interception and parameter inspection. It is not a Burp Scanner run, not automated active scanning, and not an Intruder or Repeater test. Inspection shows what is sent; it does not by itself find vulnerabilities. The vulnerabilities in Section 6 were demonstrated by the manual inputs submitted through the browser.

8. Remediation Recommendations

8.1 SQL Injection

- Use parameterized queries / prepared statements for every database query that includes user input.
- Avoid unsafe SQL string concatenation.
- Validate input (type, length, allowed values) as an additional layer, not as the primary fix.
- Apply least-privilege permissions to the database account used by the application.

8.2 Reflected XSS

- Apply context-appropriate output encoding to untrusted data.
- Use safe templating practices and auto-escaping frameworks.
- Validate input where appropriate.
- Implement a suitable Content Security Policy as defense in depth.

8.3 Secure development practices and regression testing

- Treat all user input as untrusted and follow secure coding guidance from OWASP in code reviews.
- Add automated tests that submit known injection and XSS inputs and assert safe handling.
- Retest after each fix and whenever related code changes, and re-run manual checks like those in this report.
- Use a staged approach in training: compare DVWA's Low level with its higher levels to see how safer code differs.

9. Limitations

- Testing was conducted against DVWA at the Low security level in a local lab.
- The results show weaknesses in an intentionally vulnerable training application. They do not show that any other application is vulnerable.
- Only the documented tests were performed: one SQL Injection input (plus a baseline), one Reflected XSS payload, and manual Burp request interception. No other vulnerabilities were tested, and none are claimed.
- No Burp Scanner, automated scanning, Intruder or Repeater work was performed.
- No comprehensive penetration test, source-code review or production security certification is claimed.
- The security level and the baseline (ID 1) result are not visible in the supplied screenshots. They are taken from the author's account of the testing.
- Root causes are inferred from observed behaviour, as the DVWA source code was not reviewed.
- Severity ratings are indicative and are not CVSS scores.

10. Conclusion

The assessment confirmed a SQL Injection vulnerability and a Reflected Cross-Site Scripting vulnerability in the local DVWA Low-security lab. The input `1' OR '1'='1' #` caused the SQL Injection module to return five records instead of one, and the payload `<script>alert('XSS')</script>` produced a JavaScript alert in Chrome.

Burp Suite Community Edition was used to manually intercept and inspect the GET request to the SQL Injection endpoint, which showed how the `id` and `Submit` parameters travel from the browser to the server. This inspection supports understanding of the traffic but is not a scan.

Both issues have well-established fixes: parameterized queries for SQL Injection, and context-appropriate output encoding with defense-in-depth controls for XSS. Applying the fixes and then retesting with the same inputs, together with automated regression tests, is the way to confirm the issues are closed and stay closed.

11. References

- OWASP Cheat Sheet Series, SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Cheat Sheet Series, Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- PortSwigger, Burp Suite documentation, Proxy intercept: https://portswigger.net/burp/documentation/desktop/tools/proxy/intercept
- PortSwigger, Burp Suite documentation, Intercept HTTP traffic with Burp Proxy (tutorial): https://portswigger.net/burp/documentation/desktop/tutorials/intercepting-http

Appendix A: Evidence Register

Fig.	Filename	Time shown	What it demonstrates	Used in
1	01_sql_injection_proof.png	05:16 PM, 23-10-26	DVWA SQL Injection page with ID "1' OR '1'='1' #" returning five records (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith). URL shows the encoded input.	F-01
2	02_reflected_xss_alert.png	05:17 PM, 23-10-26	Browser dialog "127.0.0.1:4280 says XSS" on the XSS (Reflected) URL containing the payload.	F-02
3	03_reflected_xss_response.png	05:17 PM, 23-10-26	DVWA Reflected XSS page; payload in the URL; response area shows "Hello". Does not alone prove execution.	F-02
4	04_burp_request_interception.png	17:33:01, 23 Oct (Burp)	Burp Proxy with Intercept on holding GET /vulnerabilities/sqli/; Inspector shows 0 query parameters.	Sec. 7
5	05_burp_sqli_parameters.png	17:35:42, 23 Oct (Burp)	Burp Proxy holding GET /vulnerabilities/sqli/?id=1&Submit;=Submit; Inspector shows query parameters id=1 and Submit=Submit.	F-01, Sec. 7

Figures 1 to 3 are the browser results; Figures 4 and 5 are the Burp captures. Not supplied: a screenshot of the baseline ID 1 result and one showing the DVWA security level.

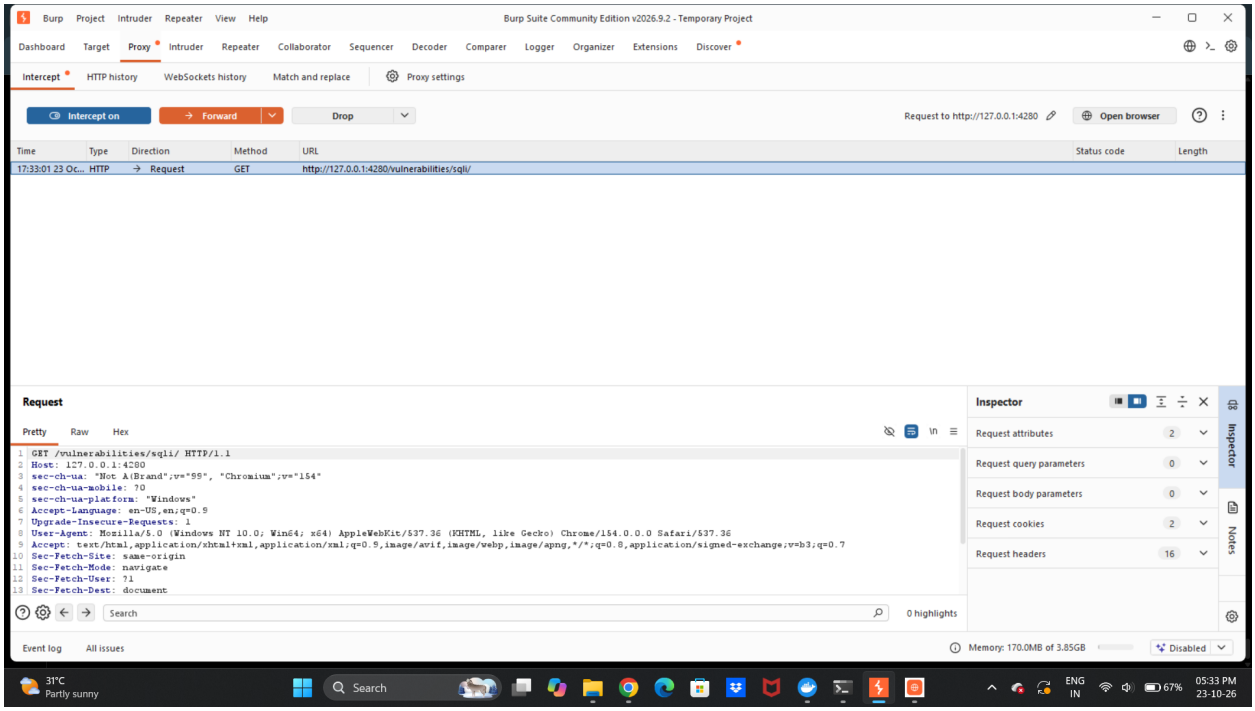


Figure 4: Burp Suite intercepted request to the SQL Injection endpoint (04_burp_request_interception.png)

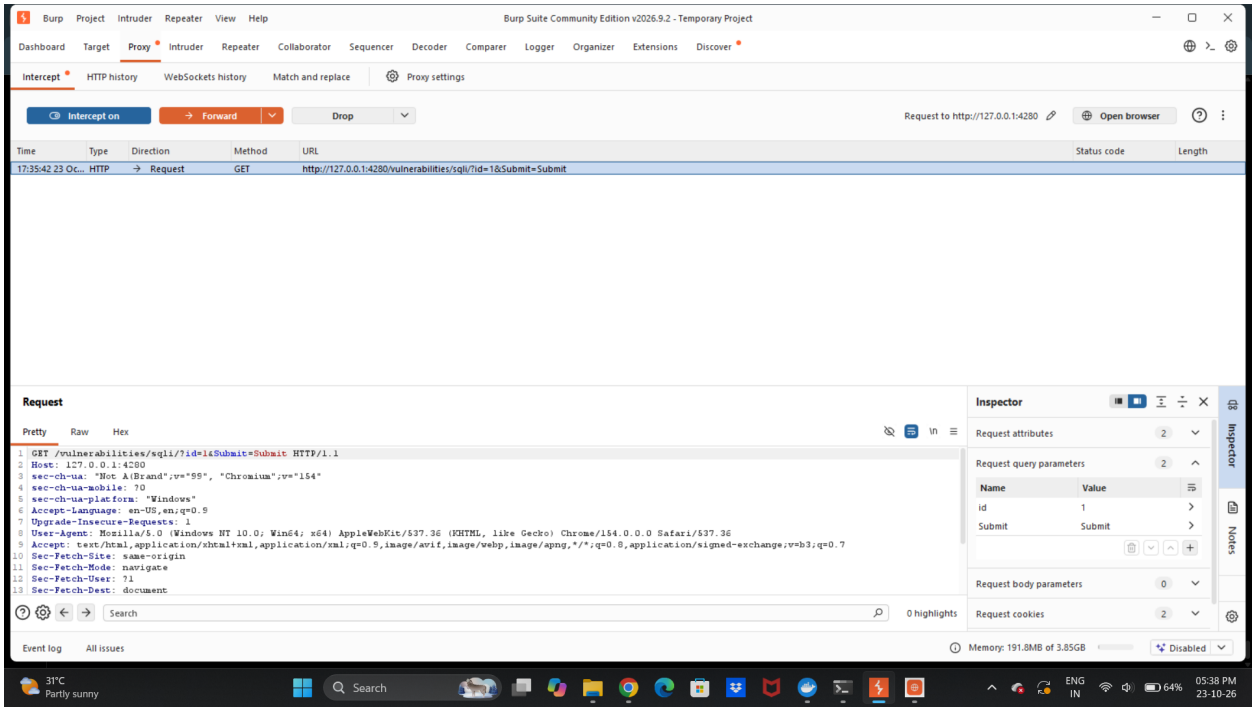


Figure 5: Burp Suite request query parameters (id and Submit) (05_burp_sqli_parameters.png)